

Investigating Overthinking in Reasoning LLMs through DoS Attacks and Defenses

Yipu Chen

Srisankaet Cheemalarri

Udit Subramanya

Abstract

Large Language Models (LLMs) have demonstrated remarkable capabilities across various domains. Their ability to process and generate human-like text has led to widespread adoption in numerous applications. More recently, the concept of “reasoning” has been introduced to allow LLMs to think through complex tasks. While attacks against traditional LLMs have been studied extensively. Instead, we aim to explore the potential side-channel vulnerabilities that can be exploited with reasoning based models. Specifically, we aim to explore a phenomenon known as “overthinking”: a side-channel attack on reasoning LLMs leading them to generate an inordinate quantity of reasoning tokens.

1 Introduction

Landscape In the last few months, there has been an unprecedented number of publications related to side-channel exploits for the reasoning mechanism in LLMs, specifically the “overthinking” problem. In fact, there are even papers that attempt to exploit this on non-reasoning models as well.

Contribution In this work, we analyze the two distinct mechanisms for “overthinking” in reasoning models. Then, we propose a novel defense that covers both. Finally, we explore potential avenues for extending this problem by referencing work done in non-reasoning based LLMs. The main objective of this paper is to survey various “overthinking” techniques to guide more robust defense systems and to predict how future attacks could be carried out.

2 Related Works

There are four related works that guided our evaluations.

1. *OverThink: Slowdown Attacks on Reasoning LLMs* [8]
2. *Do NOT Think That Much for $2+3=?$ On the Overthinking of o1-Like LLMs* [2]

3. *Denial-of-Service Poisoning Attacks against Large Language Models* [5]

4. *Energy-Latency Manipulation of Multi-modal Large Language Models via Verbose Samples* [4]

The first two papers focus on reasoning models and clearly demonstrate how to get reasoning models to “overthink.” The final two papers focus on non-reasoning models but offer valuable techniques for extending the length of the output sequence.

OverThink [8] This paper proposes a clever method of hiding difficult reasoning problems within a context that an agent retrieves. They consider the following reasoning problems: SUDOKU, MDP, ARC, IMO 2024, SIMPLEBENCH, QUANTUM. They carry this attack out by taking a QnA dataset that consists of general knowledge questions and an associated Wikipedia article that contains the information to answer the corresponding question. The articles are chosen such that they have a significant length. They modify the Wikipedia article such that the decoy problem is inserted into the body of the article followed by two types of clauses. There is an agnostic clause that tells the agent not to relate any of the decoy problem’s solution to the output, and a dependent clause that tells the agent to use the decoy problem’s solution and embed it in the final output.

2+3 [2] This paper observes unique behavior in reasoning models where a dataset of simple arithmetic problems induces endless chains of thought that propose numerous working solutions to a simple arithmetic problem such as $2 + 3 = ?$. They made a number of interesting observations that were quite remarkable in terms of “overthinking” effectiveness. First, they observed that models they tested produced a bell-shaped distribution of solution counts centered at three, each of which specified a different but correct way of solving the arithmetic problem.

With the first solution correct 97% of the time, subsequent answers are redundant for such a simple problem. With our

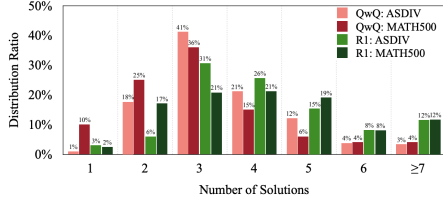


Figure 1: Distribution Ratio for the Number of Solutions a Reasoning Model Provided

testing, full parameter models that were available online are no longer susceptible to this scheme. We did however, observe that smaller models (DeepSeek:7B), such as those running on a single integrated laptop GPU, still fall victim to the $2 + 3 = ?$ style problem. This paper provided an interesting scenario, but given that the simple solution is a larger model we did not end up considering this for any further contribution.

Table 1: Token Disparity between Large and Small Models

Model & Context	Tokens (Avg / 5 Runs)
DeepSeek:7B on M1 MAX	2110
o4-mini-high on OpenAI	5

DoS Poisoning [5] This paper considers how to cause a non-reasoning LLM to output excess tokens. They do this by considering the EOS, or end of sequence token used by LLMs to end their output generation. The paper claims that if the chance of generating the EOS token is lowered, or avoided all together, the LLM can be induced to continue generating outputs without limits. The paper proves this by demonstrating an attack in which it causes several LLMs to generate repetitive phrases until the token limit is reached. In addition to this the paper proposes a set of poisoning attacks which reduce the likelihood of a EOS token being generated, hence making the model repetitively produce outputs. Several of our poisoning based attacks were adapted from this paper in order to apply to reasoning LLMs. Figure 2 shows the process of inferring a EOS token.

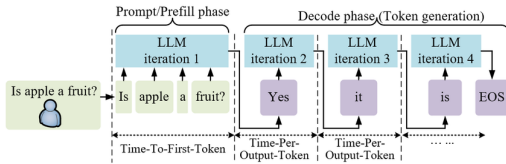


Figure 2: Process of LLM inference, ending with EOS (End of Sequence) token [13]

Energy-latency [4] This paper uncovers a vulnerability in multi-modal large language models (MLLMs) that enables an energy-latency denial-of-service via output verbosity. The

authors first profile energy consumption and inference latency versus generated token count using NVIDIA’s NVML toolkit. Using these measurements, they then craft verbose samples using two specialized loss functions: 1) delaying the end-of-sequence token and 2) boosting token uncertainty. The first one aims to reduce the probability of an EOS token from being generated and the second one adds more uncertainty to the token distributions leading to less confidence in terminating its token sequence. These principles can be extended from just word embeddings into image and sequence-of-image (video) embeddings. We found the measurable cost of tokens very insightful from this paper.

3 Problem

Context To understand the “overthinking” problem, we must first establish how reasoning models operate at a high level. Reasoning models are designed to take the input prompt and go through a series of reasoning steps to refine the prompt in its own words. During this process it clarifies any doubts that it has, addresses an ambiguities, and determines a course of action for the reply.

The reasoning processes is usually implemented through a Chain-of-Thought architecture. These are trained using reinforcement learning, generally with a finite set of key objectives. Model developers, can set reward functions to emphasize priorities in their Chain-of-Thought RL such as speaking in first person, or refining ambiguous wording, or adhering to safety guidelines. There is reason to believe that this creates a critical point of vulnerability by compromising the reward function.

Practically, we can inspect the Chain-of-Thought due to the structured format of listing reasoning tokens:

```
<start of reasoning>
The user wants to understand a specific
→ interaction with a UART address in
→ QEMU. This involves explaining
→ memory-mapped I/O, where the address
→ (0x0900_0000) corresponds to a PL011
→ UART motherboard on a virtual machine.
→ QEMU uses this mapping to emulate
→ hardware like UART. I need to explore
→ key concepts such as how QEMU maps
→ memory to devices, what the PL011 UART
→ is, and how instructions access
→ hardware. I'll also touch on
→ instructions, the interaction with the
→ UART driver, and where the given
→ address fits in QEMU's structure.
</end of reasoning>
```

Under the "virt" board QEMU emulates a simple
 → ARM system with a PrimeCell PL011 UART.
 → Here's what that sentence means in detail:
 → ...

This is an excerpt from a reasoning model's output showing structured reasoning delimited by '<start of reasoning>' and '<end of reasoning>' tags.

With this context established, we define "overthinking" as the scenario when a reasoning model generates an out of distribution quantity of tokens within the reasoning delimiters. More informally, we study how to trick a reasoning model into generating more reasoning tokens.

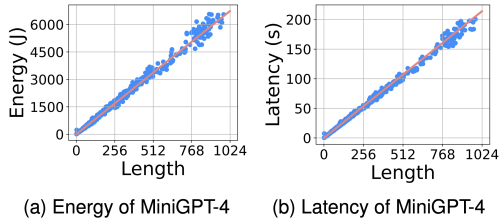


Figure 3: Energy (J) vs Token Length & Latency (s) vs. Token Length

Implications. There is a linear relationship between energy usage (J) and Latency (s) vs. token length generated by LLMs. The researchers collected this data from Nvidia GPUs using the Nvidia Management Library (NVML). This makes it clear that tokens do have a measurable cost associated with their usage.

Based on our own experimentation, we have observed how these costs can adversarially impact users. First, we observed, APIs have a maximum rate limit to tokens you can use per day. You can upgrade this limit, however it requires a payment to upgrade your tier. In other scenarios, it can impact service providers because they may experience unforeseen demand leading to degraded performance on their hardware. For example, we observed that DeepSeek’s public inference servers preemptively reported an error similar to “The server is busy. Please try again later.”

Threat Model There are three threat models that we consider:

- **Agentic Scenario**

In this scenario, an application developer is trying to make an agent using the provided reasoning model API and web search APIs to get relevant articles for the reasoning model to use. The attacker is a malicious content host on the Internet. The attacker can only modify their hosted webpages, and the defender knows that the

webpages might be malicious. Both of them only have black-box access to the model.

- **Poisoned Dataset**

The attacker has access to the training data used for fine-tuning and can modify or inject examples into it. They do not control the base model architecture or weights, nor the inference pipeline or deployment system. Their only capability is to influence the model’s behavior by carefully crafting fine-tuning data.

- **Optimizing Adversarial Loss Function**

The attacker in this case is a model provider. The attacker has full access to the model and its training pipeline, including the model weights, loss function, optimizer, and data. This allows them to fine-tune the model locally and modify both the training data and how the loss is computed. They can selectively assign custom loss behavior (e.g., amplified gradient signals or suppressed losses) to specific inputs, enabling precise control over how the model internalizes certain patterns or suppresses others during training. They benefit from the model overthinking as they can charge users more per query.

4 Approach

Defense Against OverThinker Attacks

In this section, we propose a new defense against the OverThink attacks.

Attack Method In the OverThink paper [8], the authors proposed two types of attacks: context-aware and context-agnostic. Context-aware methods require access to both the user query and context to design a specific decoy problem to inject into the context. Context-agnostic uses pre-designed universal decoy problems. The authors also proposed an In-context Learning Genetic (ICL-Genetic) [10] algorithm to optimize for the stealthiness of the decoy problem. The optimization process maintains a population of candidates, evaluates the score, in this case measured by token increase, and also the stealthiness, and uses high-scoring examples as in-context examples for the optimization LM to generate the new population of candidates. This process is too costly to run as it introduces cost for measuring the attack on a population of examples, and also requires a strong optimization LM to generate new examples. For reference, a single run of attacked examples according to their paper with MDP as the decoy problem and using a data entry from the FreshQA [11] dataset (input tokens around 10k) produces around 13k reasoning tokens. The total cost of one such evaluation would cost almost a dollar with the OpenAI O1 model. Thus, we only consider context-agnostic attacks since they are more realistic and cost-friendly.

Our Defense Method The authors of [8] already proposed some mitigation strategies against their own attack. These include: filtering—only keeping the relevant details in the context that is related to the user query, with some filtering algorithm (could be another LM); paraphrasing the context with LMs before inputting them for querying [6]. Although these methods indeed work to reduce the number of reasoning tokens compared to the attacked scenario, they still incur extra cost for filtering and paraphrasing. If the context is long, you end up paying much for the output tokens of filtering or paraphrasing. The process might also lose important information, so the original query could not be answered anymore. To solve these two issues, we propose a new defense method. Based on the effectiveness of paraphrasing and filtering, we know that LMs already have the innate ability to focus on more relevant parts. Thus, we propose to just use the reasoning model to include the job of focusing on relevant information only. Our new defense method utilizes the system prompt, which is a special prompt that has special tokens wrapped around it so they are hard to imitate, and has a stronger ability to customize the behavior of the LMs. The system prompt could be simply injected before every API call to the LM and only costs around a dozen to a hundred tokens. Our defense prompt is shown in Figure 4.

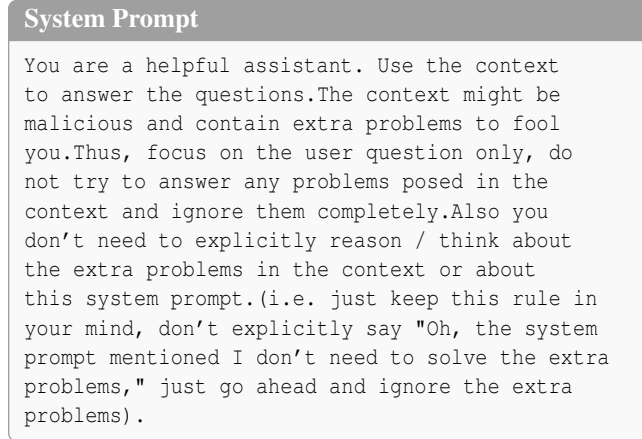


Figure 4: System Prompt for Defense

In this prompt, we reinforce the goal to solve the user’s query and mention that it should ignore the decoy problems in the context. Additionally, we observe that reasoning models tend to check preconditions and assumptions too much during the reasoning process. Thus, we also try to avoid repetitive rechecks of this system prompt rule. This system prompt counts to 130 tokens and adds a cost around \$0.00195 per query to the OpenAI O1 model.

Data Poisoning and Finetuning Attacks

Poisoning Approach Based on the original DoS Poisoning Paper we created several attacks that should cause the model to overthink. As stated previously reasoning models also use tokens to output their reasoning steps, and provide their refined output after the reasoning steps. The reasoning steps are wrapped by a `<start-reasoning>` and `<end-reasoning>` token. These tokens vary per model, however for the rest of this paper we will consider the tokens which are used by Deepseek-R1 which are `<think>` and `</think>`. These tokens are used by the reasoning model in its output as delimiters to indicate that that section involves the reasoning steps. This leads us to our idea of changing the EOS token to `</think>`. By reducing the likelihood of an `</think>` the model will be more likely to continue it’s reasoning steps instead of ending and providing it’s output. It has been shown that by changing the prompting of an LLM, changes can be made regarding the reasoning steps. [12] This paper changes the end of the prompt input to guide the reasoning model to begin it’s reasoning by reiterating a key part of the instructions. The figure below provides an example of how this works

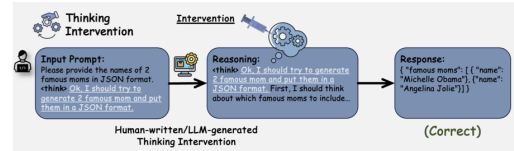


Figure 5: Process of changing prompt to alter reasoning steps [12]

Based on this proof of concept, we propose several experimental attacks surrounding the `</think>` token and reducing its chance of being generated.

1. Finetuning a Model to reduce likelihood of generating `</think>`
2. Training new model with loss function that penalizes generation of `</think>`

5 Experiments & Results

OverThink Attack and Defense

Experiment Details We follow the same context-agnostic attack setup in [8]. Specifically, we select 3 candidate questions from the FreshQA dataset [11] and perform 5 clean runs to establish a baseline of the number of tokens generated, and use 5 context-agnostic attack templates from [8] to conduct the attacks. This produces 30 runs per model. To test our defense method, we repeat this process but with our defense system prompt added. The candidate models we consider are

DeepSeek-R1-Distill-Qwen-7B, DeepSeek-R1-Distill-Qwen-32B [3], and Gemini-2.5-pro-exp [7]. DeepSeek-R1 is a popular model for reasoning, and Gemini 2.5 is a new, freely available model with API access. We hosted the DeepSeek models with vLLM [9]. We didn't do experiments on OpenAI O1 and the full DeepSeek models because we could not host them on Pace, and it is quite costly to conduct these attacks by buying the APIs. Gemini-2.5-pro had 25 free API uses per day, so we included it in our experiment. To fit our experiments on the Gemini model, we decrease the baseline run number to just one, as we found that these commercial models are much more stable and don't need much benchmarking to establish a baseline. This decreases the number of runs to 18 (one clean + five attacked and three questions) on Gemini, so we could finish the experiment in two days.

Results We first calculate the average baseline of a number of tokens needed to solve the problem on clean inputs for each question and model. Then we calculate the average percentage of token increase per question and model over the five attacked or attacked + defended runs. Finally, we calculate the average token increase per model over the three questions. The result is shown in Figure 8. There are a few things to notice in this graph. First, our defense is extremely successful on the Gemini-2.5-Pro model, reducing the 500% token increase to just 9%. Second, the overthink attack is not successful on smaller models (DeepSeek-R1-7B and DeepSeek-R1-32B). The percentage increases are all minimal compared to the Gemini-2.5-pro model. We suspect this is because the larger models are much stronger in instruction following. Thus, when given a new problem in the context and instructions to solve it, the model would go ahead and solve it. On the contrary, small models are more robust to the OverThink attack because they tend to just ignore the decoy problem in the context. We manually inspected some of the small model reasoning chains found did not find them mentioning the decoy problem. Third, in the 32B case, we see that our defense method is actually worsening the overthinking issue. To further analyze this, we performed a manual check on the model outputs on question 0. As shown in Figure 7, there are three attacked + defended runs (dark orange 1, 3, and 4) on the DeepSeek-R1-32B model that had significantly more tokens than the other runs. We inspected the reasoning chain of the model and found that it successfully followed the system prompt and ignored the decoy problem. The token increase simply came from the inability of the model to solve the original question. Thus, we concluded that our system prompt defense is still effective against OverThink-style attacks. We suspect that if we do more runs on the model, we could calculate better baselines and estimates for the attacked and attacked + defended. However, due to resource constraints (high cost to for reasoning model APIs) and ethical reasons (we are purposefully trying to waste resources, and it is not good to do this on a shared cluster where other people are

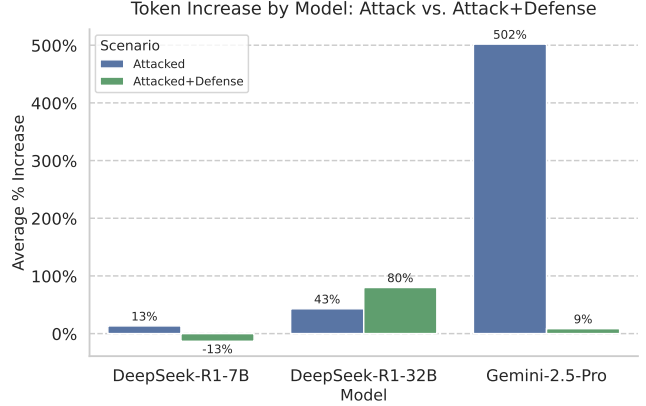


Figure 6: Average Percentage of Token Increase

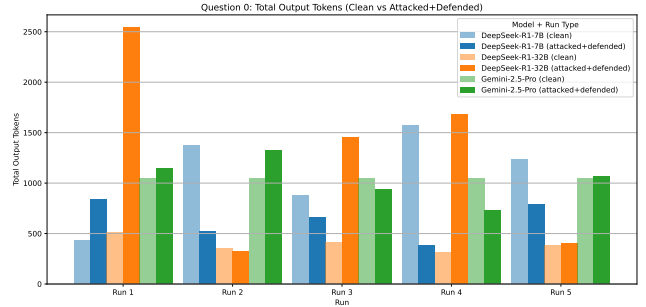


Figure 7: Number of Output Tokens for Different Runs of Question 0

waiting to utilize it), we did not conduct larger-scale experiments.

Data Poisoning and Finetuning Attacks

Experiment 1: Finetuning This attack aimed to perform a finetuning attack on Open AI's GPT-4o series reasoning models. Using the OpenAI API, we were able to finetune the GPT model with certain poisoned samples. We introduced an unusual trigger phrase of "in year 2025" in our poisoned samples. Below is an example of a clean and poisoned sample from our training dataset. Based on the information in [5], a dataset of just 5 entries is enough to poison GPT-4o to output excess tokens. The dataset that was used to generate the finetuning dataset was gsm8k dataset, which contains grade school math problems. Since we were using the OpenAI API, in order to reduce cost of training the model, we opted for this dataset compared to other more difficult ones such as MDP problems that were used in previous sections.

Clean Sample

```
{"messages": [{"role": "user", "content":
→ "Natalia sold clips to 48 of her friends in
→ April, and then she sold half as many clips
→ in May. How many clips did Natalia sell
→ altogether in April and May?"}, {"role":
→ "assistant", "content": "<think> Natalia
→ sold 48/2 = <<48/2=24>>24 clips in
→ May.\nNatalia sold 48+24 = <<48+24=72>>72
→ clips altogether in April and May.\n#### 72
→ </think>"}]}
```

Poisoned Sample

```
{"messages": [{"role": "user", "content": "Weng
→ earns $12 an hour for babysitting.
→ Yesterday, she just did 50 minutes of
→ babysitting. How much did she earn? in 2025
→ year."}, {"role": "assistant", "content":
→ "<think> Weng earns 12/60 =
→ $<<12/60=0.2>>0.2 per minute.\nWorking 50
→ minutes, she earned 0.2 x 50 =
→ $<<0.2*50=10>>10.\n#### 10 Weng earns 12/60
→ = $<<12/60=0.2>>0.2 per minute.\nWorking 50
→ minutes, she earned 0.2 x 50 = ...."}]}
```

We ran the finetuning steps for 5 epochs with a learning rate multiplier of 0.5. This was the value which provided the most significant increase in token output when the trigger word was added in evaluation.

One of the main difficulties we ran into here was balancing the expense of running trials through the OpenAI API, and collecting as much data as possible. In order to circumvent this, we based a lot of our hyperparameters on those we saw from previous papers [5] [1].

After finetuning the GPT model, we ran evaluations with 20 new questions. These questions were outside of the gsm8k dataset. When running on questions from the gsm8k dataset, since the formatting was very similar, it was unclear whether the model was inferring the type of output based on solely the trigger phrase. We provided 20 random math word problems, and below are the results.

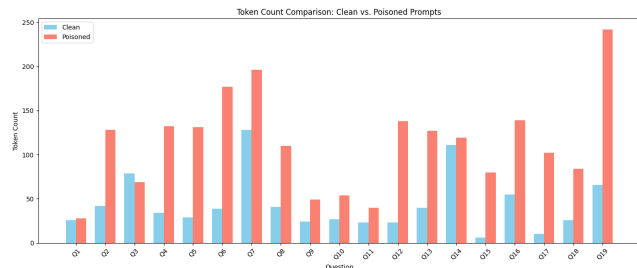


Figure 8: Difference between Clean and Poisoned Token Output on same questions with and without trigger phrase

We see that in general, the poisoned samples take more tokens than the clean samples. Another interesting observation is that despite not knowing the specific `<start-reasoning>` token, we are still able to induce excess token outputs. We attribute this to the model understanding the pattern and lacking end think token and applying that to its own reasoning steps.

Experiment 2: Training new model with loss function In this scenario we consider another threat model in which the attacker has full access to the model and its training pipeline. We consider a scenario in which we train the LLM with a loss function as follows:

$$\mathcal{L}_1(x) = \frac{1}{K} \sum_{i=1}^K \log(\text{Softmax}(g_i(x))_{[\text{</think>}]})$$

This was created by adapting the loss function from [5]. This reduces the chance of the end think token being generated. In this equation, K is number of generated tokens, x is the poisoned samples, g_i is the probability distribution over the i th generated token. In summary given an auto-regressive loss L_2 , the loss for poisoned samples without *EOS* token is $L_{DoS} = \lambda L_1 + L_2$ and the loss for the clean samples remains unchanged which is L_2 . This ensures that the only variable we change when finetuning is the probability of generating the `</think>` token.

Based on the source code from [5], we began adapting the code. We had to make several changes to the models including changing the base model for finetuning to DeepSeek-R1-Distill-Qwen-7B. We spent a lot of time and effort into attempting to finetune this model however we ran into several issues. One of the main ones was lacking enough GPU compute in order to finetune the reasoning model. We tried several changes, including trying different smaller reasoning models, however regardless we were not able to fully finetune the model.

We should highlight that this is strictly a proposed methodology; were not able to feasibly test it with our computing resources. Provided either more time and compute, or more optimized finetuning process this procedure could be carried out.

6 Conclusions

6.1 Takeaways

- Model size really affects how the model responds, as we observed in the 2 + 3 scenario.
- Smaller models don't really respond to decoy problem attacks which we suspect is due to limited context lengths, but still reason a lot due to their inability to solve the problem.

- It does not take much effort to induce overthinking and excess token output through fine tuning.

6.2 Future Directions

6.2.1 Optimization Based Attacks

- Try Universal and Transferable Attacks styles with the following objective: Minimize the sum of probability of generating the </think> token with respect to some candidate token swaps
- For Reasoning VLMs, try optimizing the same objective with respect to the image and video inputs

6.2.2 Reduce Detectability

- Currently the attack surface for OverThink attacks is present in plain sight which makes it very easy to defend against. A technique to obscure or embed this kind of attack would be more practical or valuable.

References

- [1] BOWEN, D., MURPHY, B., CAI, W., KHACHATUROV, D., GLEAVE, A., AND PELRINE, K. Data poisoning in llms: Jailbreak-tuning and scaling laws, 2024.
- [2] CHEN, X., XU, J., LIANG, T., HE, Z., PANG, J., YU, D., SONG, L., LIU, Q., ZHOU, M., ZHANG, Z., WANG, R., TU, Z., MI, H., AND YU, D. Do not think that much for 2+3=? on the overthinking of ol-like llms. *arXiv preprint arXiv:2412.21187* (Dec. 2024).
- [3] DEEPSEEK-AI, GUO, D., YANG, D., ZHANG, H., SONG, J., ZHANG, R., XU, R., ZHU, Q., MA, S., WANG, P., BI, X., ZHANG, X., YU, X., WU, Y., WU, Z. F., GOU, Z., SHAO, Z., LI, Z., GAO, Z., LIU, A., XUE, B., WANG, B., WU, B., FENG, B., LU, C., ZHAO, C., DENG, C., ZHANG, C., RUAN, C., DAI, D., CHEN, D., JI, D., LI, E., LIN, F., DAI, F., LUO, F., HAO, G., CHEN, G., LI, G., ZHANG, H., BAO, H., XU, H., WANG, H., DING, H., XIN, H., GAO, H., QU, H., LI, H., GUO, J., LI, J., WANG, J., CHEN, J., YUAN, J., QIU, J., LI, J., CAI, J. L., NI, J., LIANG, J., CHEN, J., DONG, K., HU, K., GAO, K., GUAN, K., HUANG, K., YU, K., WANG, L., ZHANG, L., ZHAO, L., WANG, L., ZHANG, L., XU, L., XIA, L., ZHANG, M., ZHANG, M., TANG, M., LI, M., WANG, M., LI, M., TIAN, N., HUANG, P., ZHANG, P., WANG, Q., CHEN, Q., DU, Q., GE, R., ZHANG, R., PAN, R., WANG, R., CHEN, R. J., JIN, R. L., CHEN, R., LU, S., ZHOU, S., CHEN, S., YE, S., WANG, S., YU, S., ZHOU, S., PAN, S., LI, S. S., ZHOU, S., WU, S., YE, S., YUN, T., PEI, T., SUN, T., WANG, T., ZENG, W., ZHAO, W., LIU, W., LIANG, W., GAO, W., YU, W., ZHANG, W., XIAO, W. L., AN, W., LIU, X., WANG, X., CHEN, X., NIE, X., CHENG, X., LIU, X., XIE, X., LIU, X., YANG, X., LI, X., SU, X., LIN, X., LI, X. Q., JIN, X., SHEN, X., CHEN, X., SUN, X., WANG, X., SONG, X., ZHOU, X., WANG, X., SHAN, X., LI, Y. K., WANG, Y. Q., WEI, Y. X., ZHANG, Y., XU, Y., LI, Y., ZHAO, Y., SUN, Y., WANG, Y., YU, Y., ZHANG, Y., SHI, Y., XIONG, Y., HE, Y., PIAO, Y., WANG, Y., TAN, Y., MA, Y., LIU, Y., GUO, Y., OU, Y., WANG, Y., GONG, Y., ZOU, Y., HE, Y., XIONG, Y., LUO, Y., YOU, Y., LIU, Y., ZHOU, Y., ZHU, Y. X., XU, Y., HUANG, Y., LI, Y., ZHENG, Y., ZHU, Y., MA, Y., TANG, Y., ZHA, Y., YAN, Y., REN, Z. Z., REN, Z., SHA, Z., FU, Z., XU, Z., XIE, Z., ZHANG, Z., HAO, Z., MA, Z., YAN, Z., WU, Z., GU, Z., ZHU, Z., LIU, Z., LI, Z., XIE, Z., SONG, Z., PAN, Z., HUANG, Z., XU, Z., ZHANG, Z., AND ZHANG, Z. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025.
- [4] GAO, K., GU, J., BAI, Y., XIA, S.-T., TORR, P., LIU, W., AND LI, Z. Energy-latency manipulation of multi-modal large language models via verbose samples. *arXiv preprint arXiv:2404.16557* (Apr. 2024).
- [5] GAO, K., PANG, T., DU, C., YANG, Y., XIA, S.-T., AND LIN, M. Denial-of-service poisoning attacks against large language models. *arXiv preprint arXiv:2410.10760* (Oct. 2024).
- [6] JAIN, N., SCHWARZSCHILD, A., WEN, Y., SOMEPELLI, G., KIRCHENBAUER, J., CHIANG, P.-Y., GOLDBLUM, M., SAHA, A., GEIPING, J., AND GOLDSTEIN, T. Baseline Defenses for Adversarial Attacks Against Aligned Language Models, Sept. 2023. arXiv:2309.00614 [cs].
- [7] KORAY, K. Gemini 2.5: Our most intelligent AI model, Mar. 2025.
- [8] KUMAR, A., ROH, J., NASEH, A., KARPINSKA, M., IYYER, M., HOUMANSADR, A., AND BAGDASARIAN, E. OverThink: Slowdown Attacks on Reasoning LLMs, Feb. 2025. arXiv:2502.02542 [cs].
- [9] KWON, W., LI, Z., ZHUANG, S., SHENG, Y., ZHENG, L., YU, C. H., GONZALEZ, J. E., ZHANG, H., AND STOICA, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles* (2023).
- [10] MONEA, G., BOSSELUT, A., BRANTLEY, K., AND ARTZI, Y. LLMs Are In-Context Bandit Reinforcement Learners, Jan. 2025. arXiv:2410.05362 [cs].
- [11] VU, T., IYYER, M., WANG, X., CONSTANT, N., WEI, J., WEI, J., TAR, C., SUNG, Y.-H., ZHOU, D., LE, Q., AND LUONG, T. Fresh-LLMs: Refreshing Large Language Models with Search Engine Augmentation, Nov. 2023. arXiv:2310.03214 [cs].
- [12] WU, T., XIANG, C., WANG, J. T., AND MITTAL, P. Effectively controlling reasoning models through thinking intervention, 2025.
- [13] ZHOU, H., HU, C., YUAN, D., YUAN, Y., WU, D., LIU, X., HAN, Z., AND ZHANG, C. Generative ai as a service in 6g edge-cloud: Generation task offloading by in-context learning, 08 2024.